ChainFunded

AN ON-CHAIN PROP-TRADING EVALUATION PLATFORM

> *A prop firm whose payout function is a smart contract cannot choose not to pay a passing trader.*

TL;DR

ChainFunded turns retail prop-firm evaluations into deterministic on-chain settlement. Traders pay USDC evaluation fees, trade simulated capital across two phases, and — if they pass without breaching any rule — receive automatic on-chain payouts from a pooled liquidity reserve. Liquidity Providers deposit USDC, mint `CFND` share tokens at 1:1 NAV, underwrite the trader-PnL spread, and earn yield from challenge fees and funded-trader firm cuts. The payout function is permissionless, re-verifies every rule on-chain against the parameters snapshotted at registration, and either transfers USDC or reverts. There is no admin path that bypasses the verification.

Document metadata

VERSION	1.0
DATE	2026-05

AUTHORS	ChainFunded Labs
CANONICAL URL	chainfunded.com/whitepaper
SOURCE CODE	github.com/Mezfinance/ChainFunded · contracts/contracts/
DEPLOYMENT	Ethereum mainnet · `ChainFundedPool` UUPS proxy
STATUS	AI audits finalised · full third-party audit in progress

---

Mainnet contract addresses

CONTRACT	ADDRESS
`ChainFundedPool` (UUPS proxy)	`0xfbdc2592f7d8ae59d1abebb88cf33b2ac70383fc`
`Timelock` (governance)	`0xdc7f1d4e9bffa8739f4b0b173700f599f7eb3ac4`
`CFGToken`	`0xde56217645c54f00258826f944f772117aca041b`
`CFGGovernor`	`0x4e8637ada57187524e30efbedd55b706f9c8be50`
`CFGDistributor`	`0x0576ff8c9291bf58c9c6a6a9623b98f05b913904`
`CFGLPRewards`	`0x0b5072a88454fa1553787bfd9cbfbe346f7d0edd`

Verify any address on Etherscan before interacting. The pool address is the canonical entry point — every other on-chain interaction either originates from the pool or reads from it (§5).

v1.0 · 2026-06-10

Contents

§ 01 Abstract

§ 02 The problem

§ 03 The thesis

§ 04 Architecture

§ 05 Trader lifecycle

§ 06 Phase progression

§ 07 LP mechanics

§ 08 NAV mechanics

§ 09 Fee economics

§ 10 Payout enforcement

§ 11 Multi-signature payouts

§ 12 Affiliate programme

§ 13 CFND token

§ 14 CFG token & DAO

§ 15 Risk surface

§ 16 Security posture

§ 17 Roadmap

§ 18 Appendix A · Threat model

§ 19 Appendix B · Notation reference

§ 01

Abstract

ChainFunded is an Ethereum protocol for the permissioned-by-rule evaluation of proprietary trading strategies. The on-chain surface consists of a single UUPS-proxied core contract, `ChainFundedPool`, which (i) collects evaluation fees in USDC, (ii) holds pooled liquidity reserves and mints the `CFNDToken` ERC-20 share token against deposited capital at a 1:1 NAV ratio, (iii) executes payouts to passing traders subject to N-of-M ECDSA signature aggregation over the protocol's authorised signer set, and (iv) emits a deterministic event log for every state transition.

The protocol's central operational property is that `claimPayout(payoutId, signatures, …)` is a permissionless function whose pre-conditions — signature threshold, signature validity, and re-verification of the trading rules snapshotted at challenge registration — are computable from the function arguments and the pool's storage. The function either transfers USDC to the trader and emits `PayoutClaimed`, or reverts with a named custom error. The contract has no admin path that bypasses these pre-conditions.

The remainder of this paper formalises three claims that follow from this construction:

1. **Settlement is monotonic and non-discretionary.** Each `claimPayout` invocation either advances the protocol's state (`settledPayouts[payoutId] := true`, USDC transferred) or reverts atomically. Replay protection via `settledPayouts` and re-entrancy protection via OpenZeppelin's `ReentrancyGuard` together rule out partial settlement.
2. **The on-chain surface is bounded and storage-stable.** The pool inherits from `Ownable2StepUpgradeable`, `ReentrancyGuard`, `PausableUpgradeable`, and `UUPSUpgradeable`. New state variables are append-only across upgrades and are checked against `contracts/storage-layout-baseline.json` in CI.
3. **All economic parameters are public state.** Fee splits (`FEE_PROTOCOL_BPS`, `FEE_PROFIT_POOL_BPS`, `MAX_AFFILIATE_BPS`), challenge tier prices (`challengeTierFee[size]`), default rules (`defaultPhase1ProfitTargetBps` et al.), and the per-CFND profit accumulator (`accProfitPerToken`) are readable from the pool at any block.

CFG token mechanics and DAO governance are out of scope for this document and are described at chainfundeddao.org.

§ 02

The problem

The retail proprietary-trading subclass that emerged in the 2020–2024 cycle operates on a fixed structural template: the trader pays an evaluation fee to access a simulated trading account governed by a written rule set, the firm attests performance through internal systems, and — conditional on a passing run — the firm settles a profit-share to the trader either as fiat off-platform or as account credit. Every state-changing operation in this loop is private to the firm: the simulated account ledger, the rule-evaluation engine, the profit calculation, the funding decision, and the eventual settlement transfer.

This template produces three identifiable failure modes that are enumerable from public regulatory and litigation records.

1 · Settlement is discretionary

The firm controls both the function and the timing of payout. Industry-public class-action filings (US 2024 — `My Forex Funds` proceedings; UK 2024 FCA action against multiple "funded-trader" providers; AU 2025 ASIC enforcement notices) document a recurring sequence: post-evaluation account closure on rule clauses ambiguously specified at registration, payout delays attributed to "compliance review", and retroactive profit-split renegotiations. The trader's leverage is contractual at best and frequently substandard against the firm's terms-of-service unilateral-amendment provisions.

2 · Capital is unverifiable

The LP base of a traditional firm is its principals plus a small private syndicate. Capital balances, segregation, and profit-and-loss are not externally auditable. The firm's rational behaviour with respect to disclosure is asymmetric: overstate capital to attract challenge buyers, understate realised P&L to suppress payout obligations. Public regulator filings have repeatedly identified this asymmetry; multiple firms shut down in 2024–2025 cited insufficient working capital to honour outstanding payouts at the moment of regulatory intervention.

3 · Regulatory perimeter is unstable

Whether a "funded-trader evaluation" with prospective profit-share constitutes an unregistered investment-contract offering is unsettled across the major financial-services jurisdictions. 2024–2025 enforcement actions in the US, UK, and Australia have produced in-

consistent remedies — in some cases fee refunds, in others account-balance freezes pending adjudication. The trader's exposure to this regulatory tail is uncompensated and unhedgeable on the firm's existing terms.

What an on-chain protocol changes

The three failure modes share a structural property: the firm's state changes are mediated by software the firm operates unilaterally. ChainFunded relocates each state change to a deterministic on-chain function with a public source. Settlement is a function call against `ChainFundedPool` (§4, §11). LP capital is `IERC20(USDC).balanceOf(pool)` minus the LP profit pool, computable at any block. The protocol's surface is bounded by the deployed contract — there is no separate "internal system" that holds material state.

This does not eliminate the regulatory question, which is external to the contract; it does eliminate the discretion and opacity that compound the regulatory exposure for an aggrieved participant.

§ 03

The thesis

The protocol's payout function is fully specified on-chain. Given a valid `payoutId`, an N-of-M signature set over the canonical EIP-712 typed message, and a snapshotted challenge state in `challenges[accountId]`, `ChainFundedPool.claimPayout` produces exactly one of two outcomes: a USDC transfer to the trader and an emitted `PayoutClaimed` event, or a transaction-level revert referencing a specific named custom error. There is no third outcome.

On-chain pre-conditions

`claimPayout(payoutId, signatures, ...)` enforces, in order:

1. **Replay protection.** `settledPayouts[payoutId]` must be `false`, else `PayoutAlreadySettled`.
2. **Signature aggregate.** `signatures.length >= signatureThreshold`, else `InsufficientSignatures`. Each signature must be from `authorisedSigners[]` and the array must be sorted ascending by recovered signer address with no duplicates, else `UnauthorizedSigner` or `DuplicateOrUnsortedSignature`.
3. **Deadline.** `block.timestamp <= deadline`, else `SignatureExpired`.

4. **Rule re-verification.** For each rule snapshotted at the challenge's `registerChallenge` call:
   - `worstDailyLoss > maxDailyLossBps` ⇒ `RuleBreachedDailyLoss`
   - `maxDrawdown > maxTotalLossBps` ⇒ `RuleBreachedTotalLoss`
   - `tradingDays < minTradingDays` ⇒ `RuleBreachedMinDays`
   - `grossProfit < profitTarget` ⇒ `RuleBreachedProfitTarget`
   - `bestDay > maxBestDayBps` ⇒ `RuleBreachedBestDay`
5. **Liquidity availability.** `lpCapitalBalance() >= traderShare`, else `InsufficientLPCapital`.

If all pre-conditions hold, `traderShare = grossProfit * traderShareBps / BPS` is transferred to the trader, the firm cut is split between `labsWallet` and the LP profit pool per `FIRM_CUT_PROTOCOL_BPS = 8000`, `settledPayouts[payoutId]` is set to `true`, and `PayoutClaimed` is emitted.

Independence of the two verification gates

The trading engine and the signer set produce a single artefact — the signed payout proof — which `claimPayout` then re-verifies against the rules registered at the time the trader paid the fee. The two computations are independent:

- The off-chain signing process is sufficient but not necessary to block a bad payout: signers will refuse to sign a proof for a breached account.
- The on-chain re-verification is necessary but not sufficient: it catches any divergence between the signed proof's claimed P&L and the rule-bound state encoded at registration.

Either gate alone produces a sound stop on a bad payout. Both gates agreeing is the precondition for a successful settlement. A compromise of all M signers does not produce a valid payout if the account's rule state is breached, because the on-chain check fails independently of the signature validity.

Liveness

The protocol does not impose a contract-level expiry on a valid proof beyond the explicit `deadline` field. A funded trader who fails to find N signers within the deadline window can re-request a payout with a new `deadline`; the pool's state for that challenge is unchanged until `settledPayouts[payoutId]` is set. There is no protocol-level mechanism that reduces a trader's earned payout amount as time passes.

§ 04

Architecture

The protocol consists of two on-chain components and two off-chain components. The on-chain components are `ChainFundedPool` (the UUPS-proxied core, deployed once and upgraded in place) and a small satellite set: `CFNDToken` (the LP share ERC-20, owned by the pool) and the canonical Circle USDC ERC-20. The off-chain components are the trading engine (which provides the simulated trading environment and computes per-trader P&L) and the payout signer set (an authorised set of EOAs that produce ECDSA signatures over EIP-712 typed messages required by `claimPayout`, `registerPhase2`, `registerFunded`, and `registerPostPayout`).

`ChainFundedPool` is the only contract that holds protocol funds. `CFNDToken` is restricted such that `mint` and `burn` are only callable by `Ownable.owner()`, which is set to the pool's address at deployment time.

Mainnet contract addresses are listed on the cover page. The pool proxy at `0xfbdc…83fc` is the canonical entry point — anything that holds USDC, mints / burns `CFND`, or settles a payout originates there.

Storage layout

`ChainFundedPool` inherits, in order, from `Ownable2StepUpgradeable`, `ReentrancyGuard`, `PausableUpgradeable`, and `UUPSUpgradeable`. The inheritance order is fixed by the storage-layout baseline; reordering bases would shift slot offsets and corrupt persisted state on upgrade.

New state variables are appended at the end of the storage block across upgrades — each new slot is positioned after the last existing one and before any event declarations, where it cannot collide with state inherited from the OZ base contracts.

`contracts/storage-layout-baseline.json` records the layout of the deployed implementation. The deploy script compares the to-be-deployed implementation's layout to the baseline and aborts on any non-appending change.

Upgrade authorisation

`_authorizeUpgrade(address newImplementation)` is `onlyOwner`. The pool owner is held by a 2-of-2 governance multisig at launch (ChainFunded Labs key + an independent guardian key). An emergency withdrawal path (`requestEmergencyWithdrawal` →

`executeEmergencyWithdrawal`) is timelocked at `EMERGENCY_DELAY = 48 hours`, providing a window during which LPs may observe a pending withdrawal and withdraw their own positions before execution.

Off-chain trust assumptions

- The trading engine is trusted to compute per-trader P&L honestly. A trading-engine fault that overstates a trader's profit produces an attested P&L that the on-chain check (which only re-evaluates rules, not raw trade execution) cannot detect. Mitigation is in operational engineering: independent signer environments, log retention, and post-hoc audit.
- Each signer is trusted to refuse to sign a proof for a breached account. Compromise of fewer than `signatureThreshold` signers is not sufficient to forge a valid proof. Compromise of `signatureThreshold` or more signers does not produce a valid payout for a rule-breached account because the on-chain check fails independently (§4, §11).

Diagram

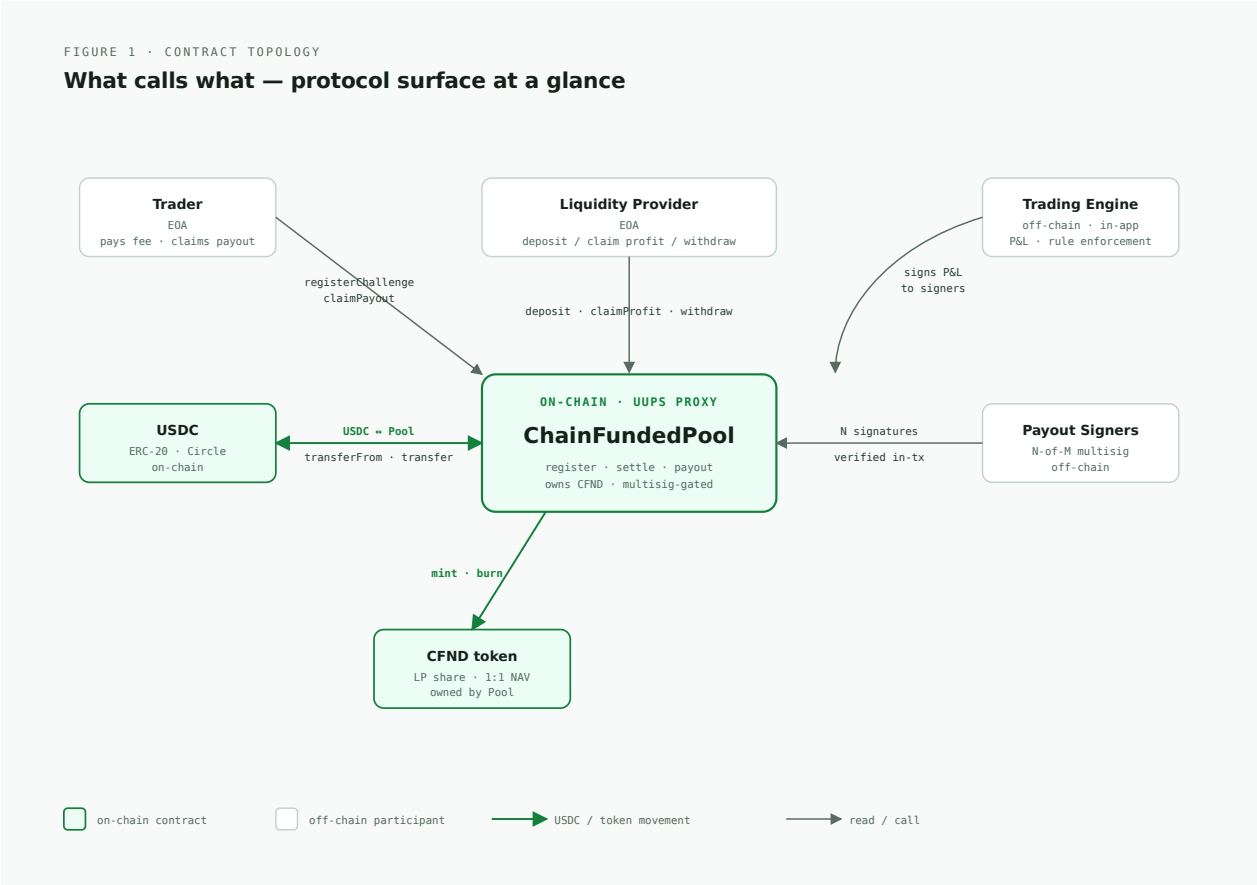


Figure 1 · Contract topology

§ 05

Trader lifecycle

A trader's interaction with the protocol consists of four discrete operations: register an evaluation challenge, trade simulated capital, advance through phase transitions verified by the multisig, and settle USDC payouts on-chain. Operations 1, 3, and 4 are on-chain transactions against `ChainFundedPool`; operation 2 is off-chain in the trading engine.

1 · `registerChallenge`

Two transactions:

```
USDC.approve(pool, fee)
pool.registerChallenge(fee, accountId, accountSize, phase, affiliate,
gasReimbursementUsdc)
```

The pool validates the fee against `challengeTierFee[accountSize]`, pulls `feeAmount` USDC, and partitions it per the on-chain splits: `FEE_PROTOCOL_BPS = 2400` to `labsWallet`, `FEE_PROFIT_POOL_BPS = 600` to the LP profit pool, an affiliate share per the §13 ladder if `affiliate != address(0)`, and the remainder to LP capital. The `Challenge` struct is written to `challenges[accountId]` with the rule snapshot (`profitTargetBps`, `maxDailyLossBps`, `maxTotalLossBps`, `minTradingDays`, `traderShareBps`, `maxBestDayBps`) frozen at this block. Subsequent owner calls to `setDefaultRules` do not modify already-registered challenges.

Reverts on `feeAmount == 0` (`InvalidAmount`), `accountSize == 0` (`InvalidAmount`), `phase != 1` (`InvalidPhase`), duplicate `accountId` (`ChallengeAlreadyRegistered`), `affiliate == msg.sender` (`SelfReferral`), `cfndToken.totalSupply() == 0` (`PoolMustHaveDeposits`), and fee/tier mismatch (`TierNotEnabled`).

2 · Trading

The trader places and closes positions through the in-app terminal against a simulated account balance. The trading engine maintains real-time per-trader state — open positions, P&L, daily loss, drawdown, trading-day count, best-day percentage — and rejects trades that would cross a hard limit. The pool has no visibility into trade execution; its next interaction with the challenge is the phase-transition or payout call.

This separation is by construction: putting trade execution on-chain would impose latency and gas costs that exceed the operational utility, given that the contract's settlement guarantee does not depend on on-chain trade execution.

3 · Phase transitions

When a trader meets the phase profit target without breaching any loss or trading-day rule, the trading engine assembles an EIP-712 typed message and forwards it to the signer set. Each signer independently re-derives the trader's P&L from the trade ledger and signs an ECDSA proof if the rules verify. The trader collects N (where `N = signatureThreshold`, currently `3`) of `M` signatures and submits:

```
pool.registerPhase2(accountId, signatures)    // Phase 1 → Phase 2
pool.registerFunded(accountId, signatures)    // Phase 2 → Funded
```

Both calls verify the signature aggregate with the same logic used in `claimPayout` (§11) and advance `challenges[accountId].phase`.

4 · `claimPayout`

A funded trader issues a payout request off-chain, signers sign a proof of the asserted P&L, and the trader submits:

```
pool.claimPayout(payoutId, signatures, ...)
```

The function re-runs the rule check against the snapshotted `Challenge` struct, splits the asserted profit by `traderShareBps` (default `8000`) into `traderShare` and `firmCut`, transfers the trader's share, splits the firm cut between `labsWallet` and the LP profit pool, sets `settledPayouts[payoutId] = true`, and emits `PayoutClaimed`. The function is replay-protected, re-entrancy- guarded, and pause-respecting.

Once mined, the settlement is terminal. The pool exposes no function that reverses a settled payout, no admin path that clawbacks transferred USDC, and no upgrade path that would do either without breaking the storage-layout baseline.

Diagram

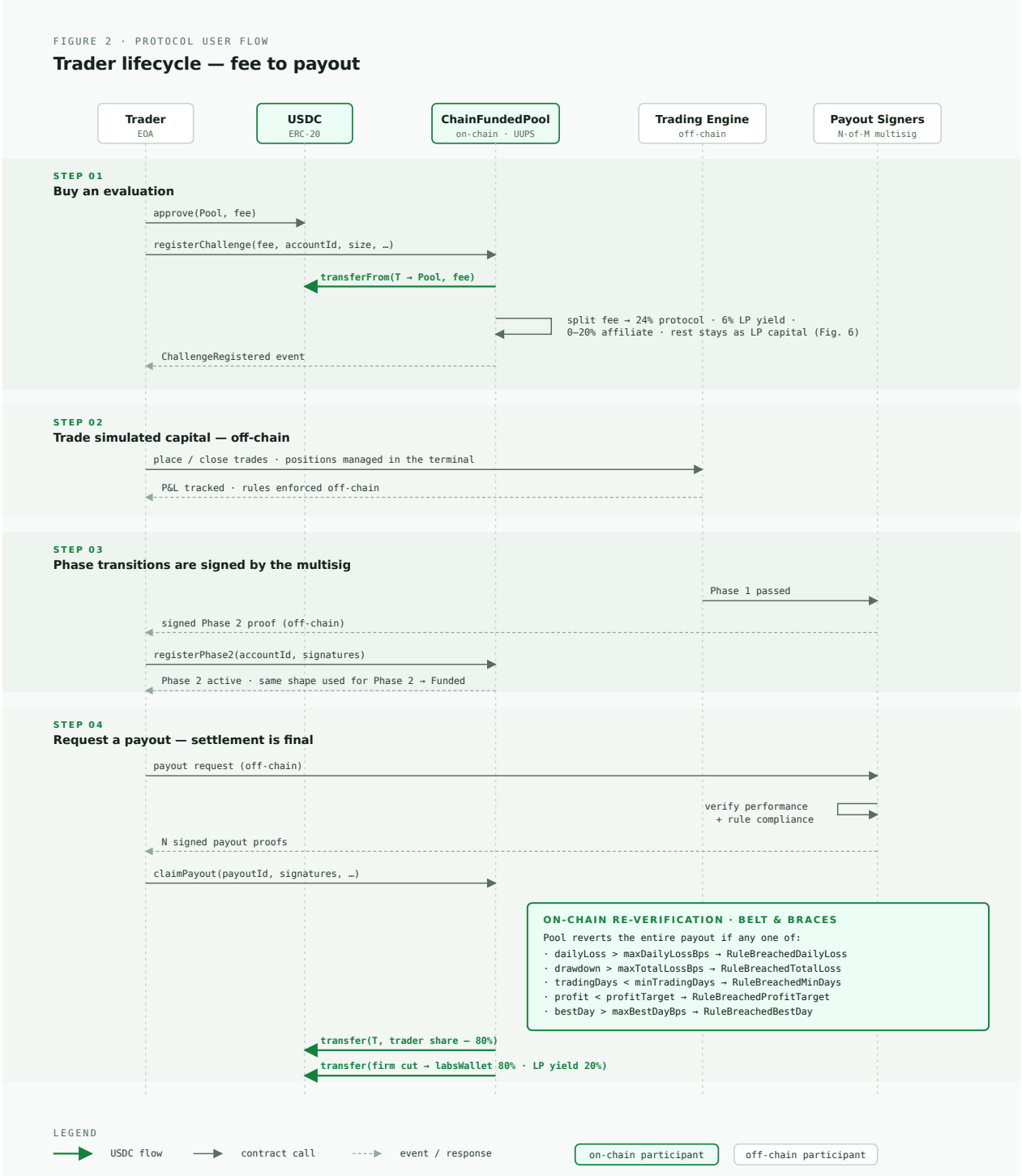


Figure 2 · Trader lifecycle — fee to payout

§ 06

Phase progression

Each `Challenge` struct in `challenges[accountId]` encodes its state through two fields: `phase` (`uint8`, range `1–3`) and `settled` (`bool`). The combinations enumerate the protocol's finite challenge state set:

STATE	PHASE	SETTLED	ENTERED BY	EXITED BY
Phase 1 active	`1`	`false`	`registerChallenge`	`registerPhase2` (→ Phase 2 active) · off-chain breach (→ Failed)
Phase 2 active	`2`	`false`	`registerPhase2`	`registerFunded` (→ Funded active) · off-chain breach (→ Failed)
Funded active	`3`	`false`	`registerFunded`	`claimPayout` (→ Settled) · off-chain breach (→ Failed)
Settled	`*`	`true`	`claimPayout`	`registerPostPayout` (→ new Phase 1 active for the same trader)
Failed	`*`	`true`	off-chain rule breach (no on-chain transition; the engine ceases signing)	terminal

`registerPhase2`, `registerFunded`, `claimPayout`, and `registerPostPayout` each accept an N-of-M signature aggregate over an EIP-712 typed message containing `(accountId, newPhase, trader, deadline, ...)`. The pool verifies signature validity, threshold, sorted-uniqueness, and deadline against the same signer set used by `claimPayout` (§11). Failure modes match the `claimPayout` revert table for signature validation; phase-transition calls additionally revert `InvalidPhase` if the requested transition is not a valid `(currentPhase, newPhase)` adjacency.

Re-registration loop

`registerPostPayout(settledAccountId, freshAccountId, ...)` allows a trader to roll a settled challenge into a fresh Phase 1 challenge for a new accountId without paying a new fee. The function requires:

- `challenges[settledAccountId].settled == true`

- `challenges[settledAccountId].trader == msg.sender`
- `challenges[freshAccountId].createdAt == 0` (`FreshAccountAlreadyRegistered` otherwise)
- `signatures.length >= signatureThreshold`

This is the only edge in the state machine that returns from a terminal state, and it produces a new `Challenge` record at a new accountId rather than mutating the settled record. The settled challenge's storage slot is preserved for downstream event-replay correctness.

Diagram

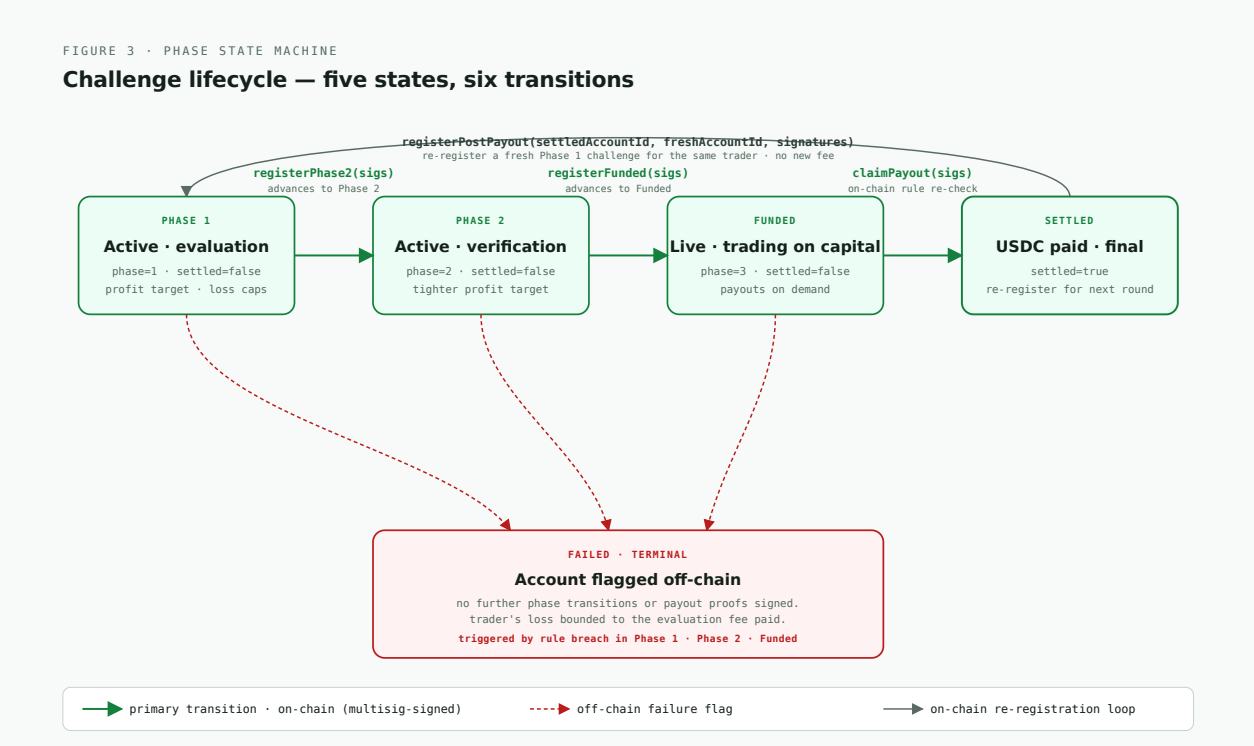


Figure 3 · Challenge phase state machine

§ 07

LP mechanics

A liquidity provider deposits USDC into `ChainFundedPool` and receives `CFNDToken` (`CFND`) — an ERC-20 share token whose balance encodes a pro-rata claim against (i) the pool's LP capital reserve and (ii) the per-CFND share of the LP profit pool. `CFND` is owned and

minted exclusively by the pool; `mint(to, amount)` and `burn(from, amount)` are restricted to `Ownable.owner()` which is set to the pool's address at deployment.

Deposit

```
USDC.approve(pool, amount)
pool.deposit(amount, gasReimbursementUsdc)
```

The pool pulls `amount` USDC, mints `amount` CFND to the LP (1:1 at NAV), and updates `lastDepositBlock[lp] = block.number`. The `lastDepositBlock` mapping is consulted by `withdraw`: withdrawals from the same block as the depositor's most recent deposit revert with `SameBlockDeposit`. This blocks the deposit-then-withdraw flash pattern that would otherwise permit a same-block round-trip through the per-CFND profit accumulator.

Reverts: `InvalidAmount` (`amount == 0`), `PoolMustHaveDeposits` (n/a — first deposit creates the deposit-pool invariant; this revert is on `registerChallenge` when the pool has zero LPs).

Yield accrual

Two USDC streams accrue to the LP profit pool, separately from the LP capital reserve:

- `(feeAmount * FEE_PROFIT_POOL_BPS) / BPS = 6%` of every challenge fee credited at `registerChallenge`.
- `(firmCut * (BPS - FIRM_CUT_PROTOCOL_BPS)) / BPS = 20%` of every funded-trader payout firm cut credited at `claimPayout`.

The pool tracks this credit via `accProfitPerToken` — a fixed-point per-token accumulator updated whenever the profit pool gains or loses USDC. Each LP carries `rewardDebt[lp]` = `balance × accProfitPerToken` at the time of their last state-changing operation (deposit, withdraw, claim, `transferCfnd`). Pending profit at any time is:

```
pendingProfit(lp) = balance(lp) × accProfitPerToken − rewardDebt(lp)
```

This is the per-share-accumulator pattern used in MasterChef and its derivatives; it is `O(1)` per claim and exact under fixed-point arithmetic.

Claim profit

```
pool.claimProfit(gasReimbursementUsdc)
```

Transfers `pendingProfit(msg.sender)` USDC to the LP and resets `rewardDebt[msg.sender]` to the new `balance × accProfitPerToken`. CFND balance is unchanged. The LP capital reserve is unchanged. Reverts on `NoProfitToClaim` if `pendingProfit == 0`.

Withdraw capital

```
pool.withdraw(cfndAmount, gasReimbursementUsdc)
```

Burns `cfndAmount` CFND from the LP and transfers `cfndAmount × NAV` USDC. The function first auto-claims any pending profit (so `claimProfit` is not required as a prerequisite), then computes and transfers the capital share at the current NAV. Reverts: `InsufficientCFND` (`balance < cfndAmount`), `NoCapitalAvailable` (`lpCapitalBalance() == 0`), `SameBlockDeposit` (`lastDepositBlock[lp] == block.number`).

Diagram

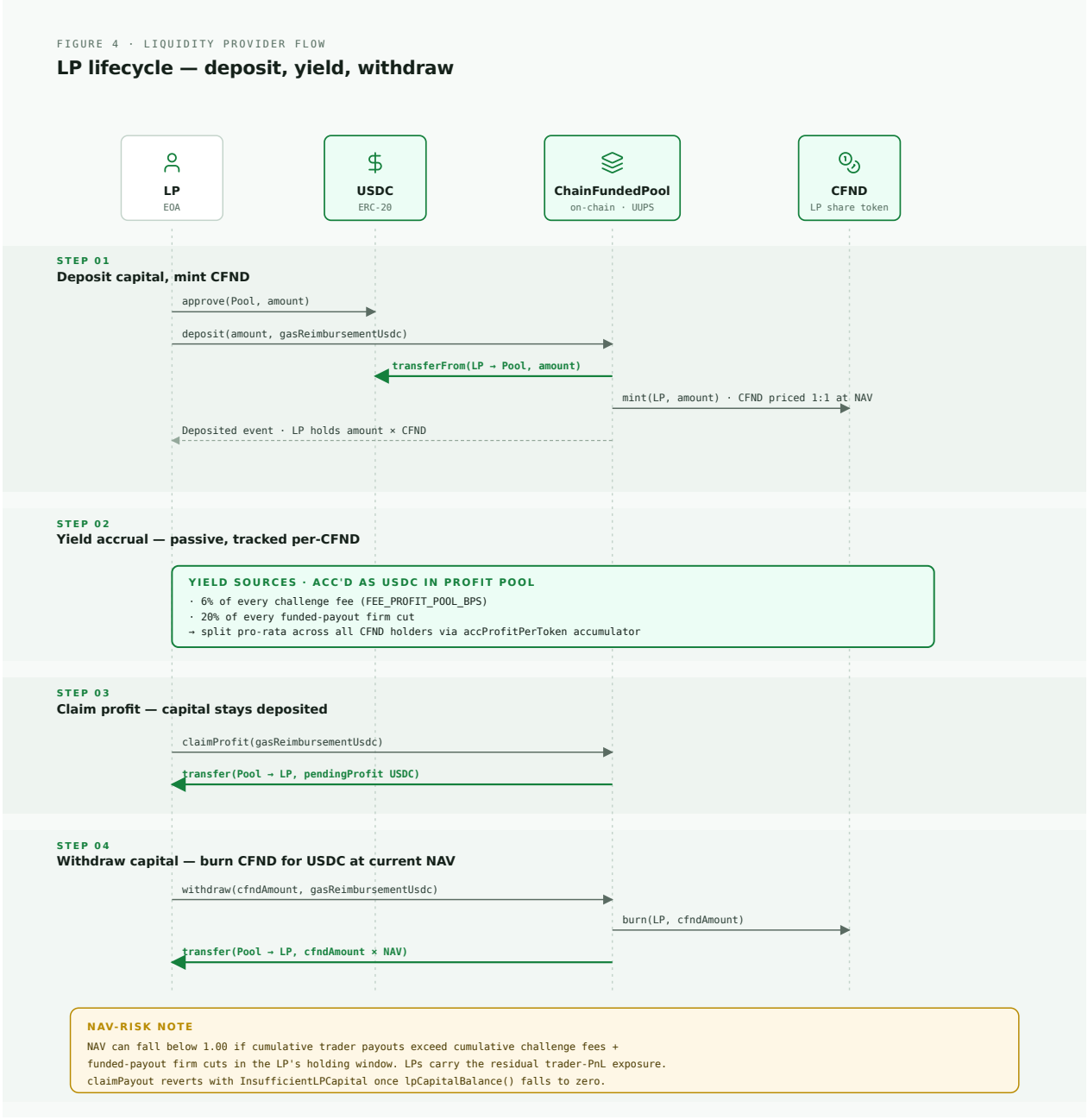


Figure 4 · LP lifecycle

§ 08

NAV mechanics

The CFND net asset value is computed deterministically from public pool state at any block:

```
NAV(t) = ( totalUSDC(t) − profitPoolBalance(t) ) / cfndTotalSupply(t)
```

where `totalUSDC(t)` = `IERC20(USDC).balanceOf(pool)` at block t, `profitPoolBalance(t)` = `pool.profitPoolBalance()`, and `cfndTotalSupply(t)` = `cfndToken.totalSupply()`.

Excluding `profitPoolBalance` from the numerator is the mechanical guarantee of `claimProfit` NAV-neutrality: `claimProfit` debits both the pool's USDC balance and the profit pool by the same amount, so the ratio is invariant.

Operations that change NAV

OPERATION	NAV IMPACT	MECHANISM
`registerChallenge`	NAV ↑ by the LP-capital share of the fee	LP-capital share = `feeAmount × (BPS − FEE_PROTOCOL_BPS − FEE_PROFIT_POOL_BPS − affiliateRateBps) / BPS`. For an unreferred trade at default constants this is 70% of the fee.
`claimPayout` (trader share)	NAV ↓ by `traderShare`	`traderShare` is transferred out of the pool's USDC balance; LP capital reserve = `totalUSDC − profitPool` decreases.
`claimPayout` (LP-yield slice of firm cut)	NAV ↑ by 20% of firm cut	This slice credits the profit pool, not capital, so it cancels out of the NAV numerator (numerator unchanged) — but `totalUSDC` rises, so the numerator's preserving property holds without exception.

Aggregated over time, NAV is a martingale on (cumulative challenge fees + firm-cut LP slice − cumulative trader payouts). For periods in which the sum is positive, NAV monotonically appreciates. For periods in which it is negative, NAV depreciates — bounded below by `0` because the pool cannot transfer more USDC than it holds. `claimPayout` reverts with `InsufficientLPCapital` if the requested `traderShare` exceeds `lpCapitalBalance() = totalUSDC − profitPool`.

Worked examples

The relationship between `totalUSDC`, `profitPool`, NAV, and per-CFND yield is easiest to see with concrete numbers. Both scenarios assume `cfndTotalSupply = 1,000,000 CFND`.

Healthy state. Pool has accrued fees beyond the original LP deposits, with $50,000 of that earmarked as LP yield:

```
totalUSDC        = $1,050,000
profitPool       =    $50,000
lpCapitalBalance = $1,000,000   (totalUSDC − profitPool)
NAV              =    $1.00     ($1M / 1M CFND)
profit per CFND  =    $0.05     ($50k / 1M CFND)
total claim/CFND =    $1.05     (NAV + earned yield)
```

NAV is unchanged from launch — LP *capital* is whole. The $0.05 per CFND of yield sits in the profit pool and is claimable via `claimProfit()` without moving NAV (both numerator and denominator of the NAV equation are independent of the profit-pool withdrawal).

Stress state. A large funded-trader payout has eaten into LP capital faster than fees have replenished it:

```
totalUSDC        = $1,000,000
profitPool       =    $50,000
lpCapitalBalance =   $950,000
NAV              =    $0.95     ($950k / 1M CFND)
profit per CFND  =    $0.05
total claim/CFND =    $1.00     (NAV + earned yield = breakeven)
```

NAV has depreciated; LP capital backing per CFND is below $1. The accumulated yield brings each CFND back to breakeven on a total-claim basis, but withdrawing capital now would crystallise the NAV loss. This is the trader-PnL exposure LPs underwrite by participating in the pool.

Diagram

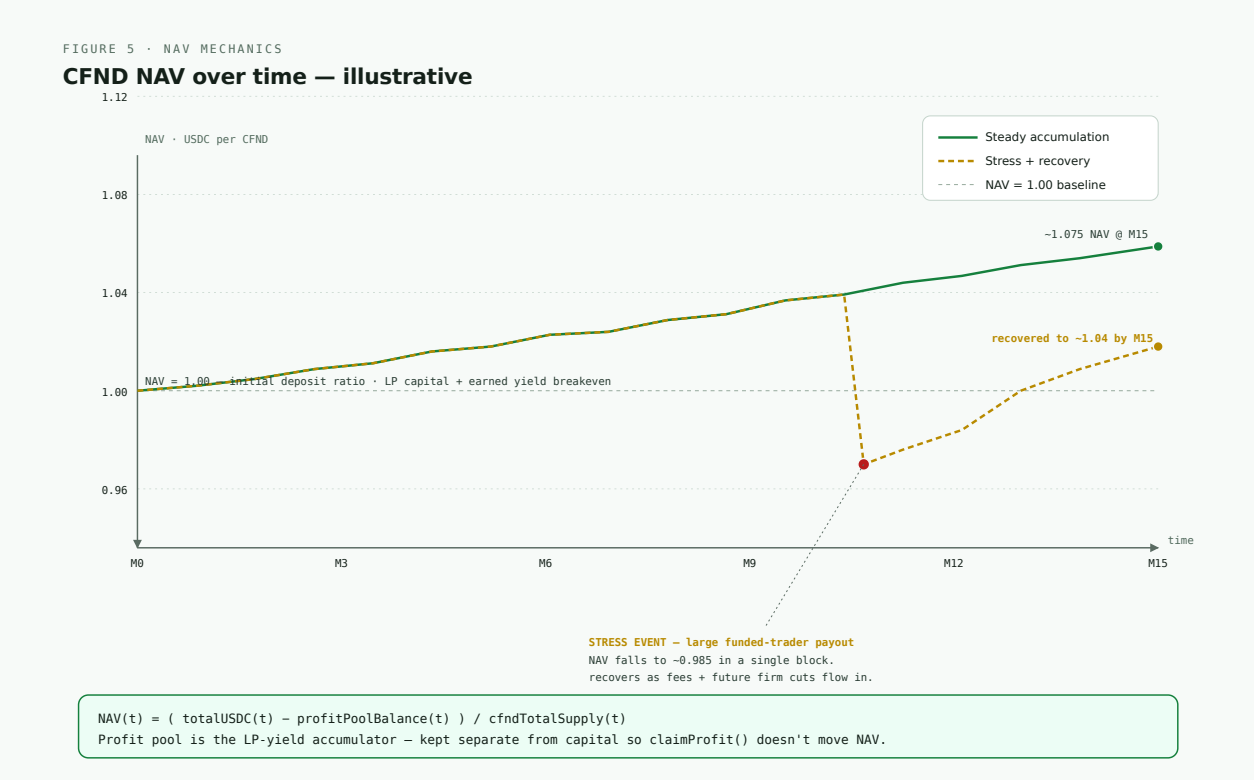


Figure 5 · CFND NAV over time — illustrative

Properties

- **NAV is observable.** Any participant can compute current NAV from `IERC20(USDC).balanceOf(pool)`, `pool.profitPoolBalance()`, and `cfndToken.totalSupply()` using a single multicall.
- **NAV is monotone in fee inflows.** A `registerChallenge` call cannot decrease NAV; a successful `claimPayout` cannot increase it (the LP-yield slice credits a profit-pool slot outside the NAV numerator).
- **NAV does not encode unrealised trader P&L.** Because trade execution is off-chain (§6), the pool has no view of unsettled funded-trader gains until `claimPayout` is invoked. NAV reflects only on-chain state: the protocol's actual USDC balance net of LP yield earmarks, divided by the supply of CFND.

LPs are taking a long position on the aggregate distribution of trader outcomes net of fee revenue and firm cuts. The exposure is denominated in USDC and computable from public state at every block. There is no contract-side mechanism that obscures this exposure.

§ 09

Fee economics

The challenge fee paid at `registerChallenge(feeAmount, ...)` is partitioned in the same transaction. Three constants and one mapping govern the partition:

IDENTIFIER	VALUE	DESTINATION
`FEE_PROTOCOL_BPS`	`2400` (24%)	`labsWallet` (protocol revenue)
`FEE_PROFIT_POOL_BPS`	`600` (6%)	LP profit pool (yield to CFND holders)
`MAX_AFFILIATE_BPS`	`2000` (20%)	hard ceiling on affiliate share
`affiliateRateBps(monthlyVolume)`	`0`–`2000` (per ladder)	affiliate wallet, when `affiliate != address(0)`
(residual)	`5000`–`7000`	LP capital reserve (backs payouts)

Constants are immutable in the implementation; modifying them requires a UUPS upgrade and triggers the storage-layout baseline check in CI. The affiliate rate is the single variable component, set per-call by the contract's `_affiliateRateBps` function against the affiliate's rolling-30-day volume; see §13 for the ladder configuration.

Funded-trader payout splits

`claimPayout` partitions the asserted profit between the trader and the firm using the per-challenge `traderShareBps` recorded at `registerChallenge` (default `8000` — 80% to trader). The remaining 20% (the firm cut) is then split:

IDENTIFIER	VALUE	DESTINATION
`FIRM_CUT_PROTOCOL_BPS`	`8000` (80% of firm cut)	`labsWallet`
(residual)	`2000` (20% of firm cut)	LP profit pool

Composing: of every USDC of trader profit, the trader receives 80%; `labsWallet` receives 80% × 20% = 16%; the LP profit pool receives 20% × 20% = 4%.

Worked example — a $50 challenge fee

For a trader buying the $2,500-tier challenge at the $50 launch-sale price, with no referring affiliate:

```
feeAmount                = $50.00
protocol (24%)           = $12.00  → labsWallet
LP yield pool (6%)       =  $3.00  → accProfitPerToken updated
affiliate (0%)           =  $0.00  → no transfer (affiliate=address(0))
LP capital reserve (70%) = $35.00  → stays in pool
```

For the same trader referred by a Tier-3 affiliate (12%):

```
feeAmount                = $50.00
protocol (24%)           = $12.00  → labsWallet
LP yield pool (6%)       =  $3.00  → accProfitPerToken
affiliate (12%)          =  $6.00  → affiliate wallet
LP capital reserve (58%) = $29.00  → stays in pool
```

The affiliate share is the only variable component; the protocol and LP-yield slices are fixed by `FEE_PROTOCOL_BPS` and `FEE_PROFIT_POOL_BPS`. Worst case for LP capital is a maximum-tier affiliate at the `MAX_AFFILIATE_BPS = 2000` cap, leaving 50% of the fee as LP-capital reserves.

Diagram

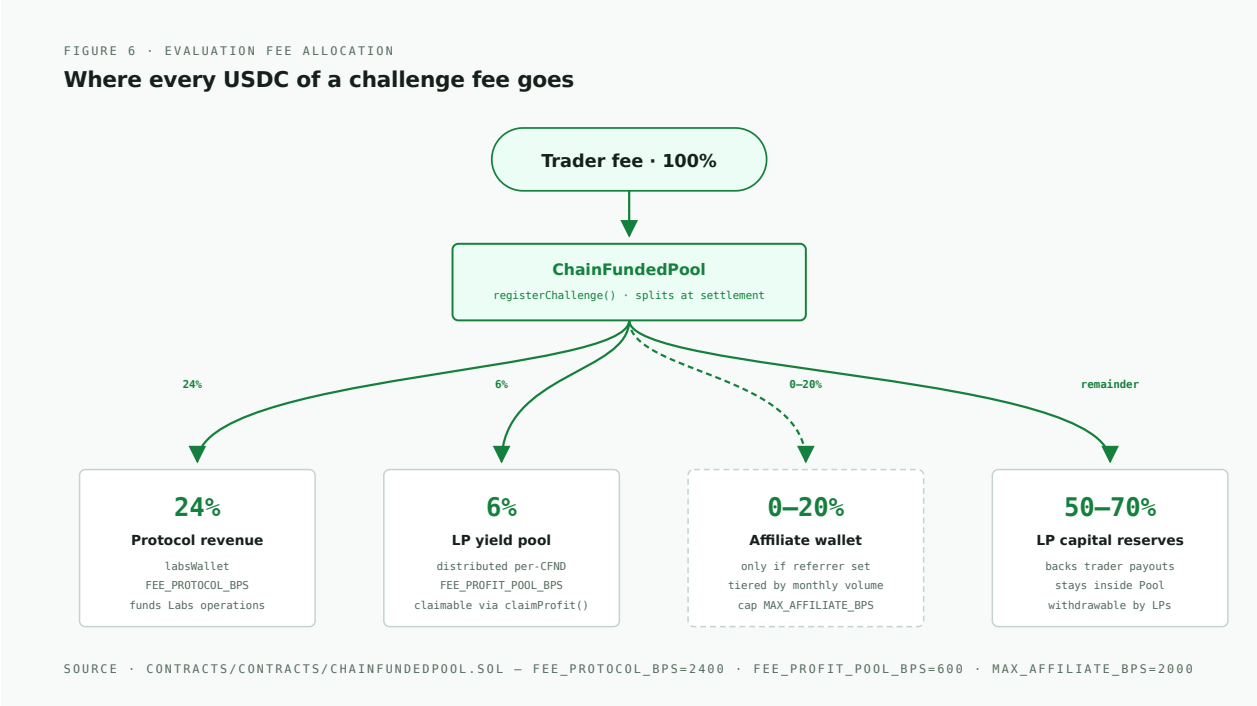


Figure 6 · Where every USDC of a challenge fee goes

Properties

- **Allocation is atomic.** Fee partitioning happens inside `registerChallenge` in a single transaction; there is no separate "settle" step that could leave the splits in an inconsistent state. The same property holds for firm-cut splits inside `claimPayout`.
- **Splits are public and grep-able.** Each split is a public constant or a publicly-readable mapping (`challengeTierFee`, `affiliateTierThresholds`, `affiliateTierRates`).
- **Tier prices are owner-configurable; splits are not.** The owner can update `challengeTierFee[size]` per tier via `setChallengeTiers`, but the percentage allocation (`FEE_PROTOCOL_BPS` etc.) is hard-coded into the implementation.

The 24%-protocol / 6%-LP-yield ratio is front-loaded by design: protocol operating costs (signer infrastructure, audits, ongoing development) are funded at challenge purchase, not at payout settlement. This decouples revenue from trader-pass rates, which is the relevant property for sizing protocol-level runway.

§ 10

Payout enforcement

The `claimPayout` function is the protocol's settlement entry point. It is `external nonReentrant whenNotPaused`, takes a payout identifier, an aggregated signature set, and a struct describing the asserted trading result, and produces — atomically — either a USDC transfer plus a `PayoutClaimed` event, or a transaction-level revert.

Function pre-conditions

The function reverts on any of the following conditions before any state mutation:

PRE-CONDITION VIOLATED	CUSTOM ERROR
`settledPayouts[payoutId] == true`	`PayoutAlreadySettled`
`block.timestamp > deadline`	`SignatureExpired`
`signatures.length < signatureThreshold`	`InsufficientSignatures`
any signer ∉ `authorisedSigners[]`	`UnauthorizedSigner`
signatures unsorted or contain a duplicate signer	`DuplicateOrUnsortedSignature`
`worstDailyLoss > challenge.maxDailyLossBps`	`RuleBreachedDailyLoss`
`maxDrawdown > challenge.maxTotalLossBps`	`RuleBreachedTotalLoss`
`tradingDays < challenge.minTradingDays`	`RuleBreachedMinDays`
`grossProfit < challenge.profitTarget` (where applicable)	`RuleBreachedProfitTarget`
`bestDay > challenge.maxBestDayBps`	`RuleBreachedBestDay`
`lpCapitalBalance() < traderShare`	`InsufficientLPCapital`

Each error is a distinct named custom error type; the failing condition is identifiable from the transaction receipt without reference to off-chain logs.

Independence of off-chain attestation and on-chain re-verification

The signed payout proof is produced off-chain by N of M signers, each of whom independently re-derives the trader's P&L from the trade ledger and re-checks the rule set. The signers' role is attestation: they produce a typed message hash signed with their private keys, asserting that the supplied P&L is consistent with the asserted account state.

`claimPayout` then re-runs the same rule evaluation on-chain, against the rule parameters snapshotted in `challenges[accountId]` at the time the trader paid the fee. The on-chain evaluation does not consume or trust the signers' assertion of P&L beyond what is inside the message hash they signed. If the signers attest to a P&L that, when applied to the snapshotted rule set, would breach a rule, the contract reverts irrespective of signature validity.

This independence rules out the failure mode in which a compromised signer set produces a valid signature aggregate over a fraudulent P&L: such an aggregate either does not breach the snapshotted rules (in which case the payout is correct by construction) or does breach them (in which case the on-chain check catches it).

Out of scope

`claimPayout` does not arbitrate disputes about the underlying trade ledger. It does not verify market data correctness, exchange uptime, or the temporal ordering of off-chain trades. These are operational concerns of the trading engine and the signer set; the contract's guarantee terminates at the boundary "the supplied P&L, re-evaluated against the registered rules, does not breach any of them".

A trading-engine bug that mis-records a trade as profitable when it was unprofitable would produce a P&L that the on-chain check accepts. Mitigating this class of failure is the subject of §17 (security posture).

Diagram

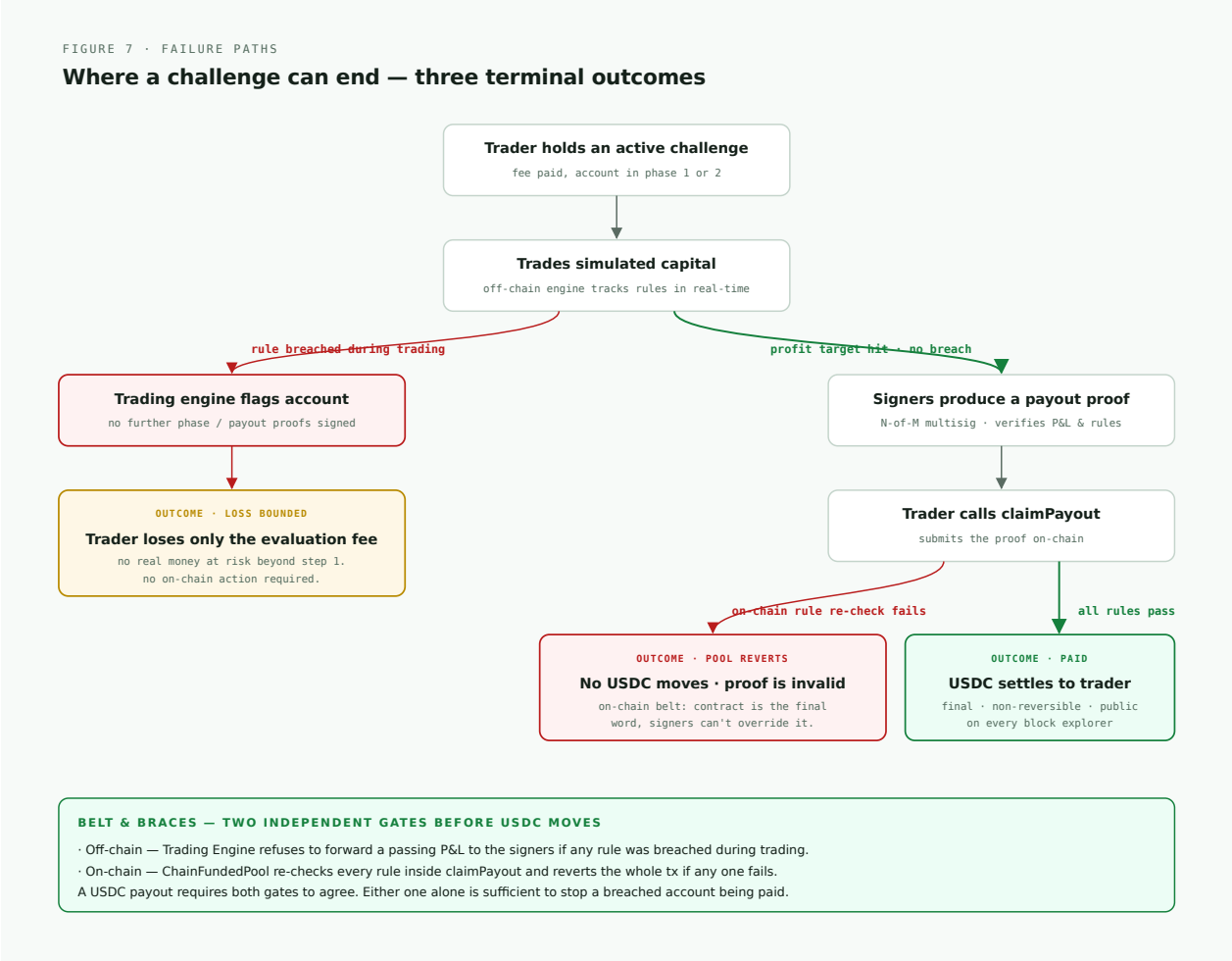


Figure 7 · Failure paths · three terminal outcomes

§ 11

Multi-signature payouts

`ChainFundedPool` maintains an `authorisedSigners` set (`mapping(address => bool)`) and a `signatureThreshold` (default `3` of `M=5` at launch). Every state-changing payout-related function — `registerPhase2`, `registerFunded`, `claimPayout`, `registerPostPayout` — requires N ECDSA signatures from authorised signers over an EIP-712 typed message hash whose fields include `(accountId, payoutId, claimedRules, deadline, chainId, contractAddress)`.

Set management

Three owner-only functions mutate the signer set:

FUNCTION	PRE-CONDITION	POST-CONDITION
`addSigner(address)`	`!authorisedSigners[addr]` (else `DuplicateSigner`) · `addr != address(0)` (else `InvalidAddress`)	`authorisedSigners[addr] = true`; `signerCount += 1`
`removeSigner(address)`	`authorisedSigners[addr]` (else `NotASigner`) · `signerCount − 1 ≥ signatureThreshold` (else `WouldBreachThreshold`)	`authorisedSigners[addr] = false`; `signerCount −= 1`
`setSignatureThreshold(uint256)`	`newThreshold > 0` · `newThreshold ≤ signerCount` (else `InvalidThreshold`)	`signatureThreshold = newThreshold`

The `WouldBreachThreshold` guard is the central safety property: it is impossible to remove a signer in a way that leaves the remaining set unable to produce a valid signature aggregate.

Signature validation

Each signature is recovered via `ECDSA.recover(messageHash, signature)` (OpenZeppelin v5). The pool requires:

```
sigs.length >= signatureThreshold                       (else
InsufficientSignatures)
recovered_addr ∈ authorisedSigners[]   for each sig     (else UnauthorizedSigner)
sigs sorted ascending by recovered_addr, no duplicates  (else
DuplicateOrUnsortedSignature)
block.timestamp <= deadline                             (else SignatureExpired)
```

The sorted-by-recovered-address requirement makes duplicate detection `O(N)` and removes ambiguity in signer-aggregate canonicalisation: the same (rule-state, signer-set) pair produces exactly one valid `signatures[]` ordering.

Independence of attestation paths

Each signer operates an independent verification environment. The protocol does not specify a shared signing ceremony; each signer's stack is isolated by design. Compromise of one signer key gives the attacker zero ability to forge a payout proof unless they can also compromise `signatureThreshold − 1` additional keys.

The on-chain re-verification at `claimPayout` (§4, §11) is independent of signer compromise. A complete signer-set compromise that produces a valid signature aggregate over a fraudulent P&L either does not breach the on-chain rules (in which case the payout is correct under the rules at registration) or does breach them (in which case the contract reverts on the matching `RuleBreached*` error).

Diagram

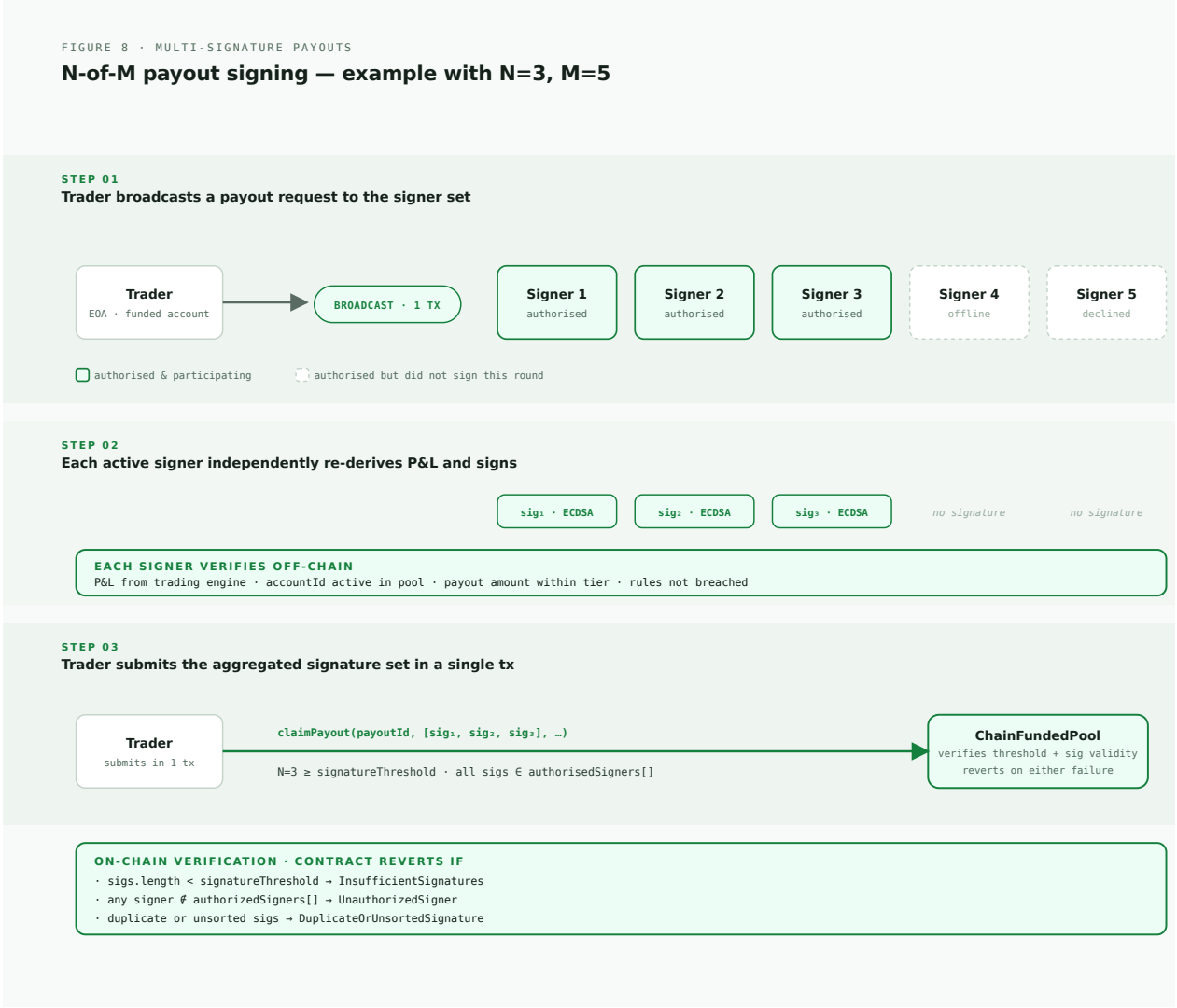


Figure 8 · N-of-M payout signing

Threshold parameter selection

For an N-of-M signer set, the protocol tolerates up to `k = M − N` simultaneous signer outages before payout liveness is impaired, and requires the attacker to compromise at least `N` keys to forge a valid signature aggregate.

CONFIGURATION	N-OF-M	OUTAGE TOLERANCE (`M − N`)	MIN KEYS TO COMPROMISE
Launch	3-of-5	2	3
Hardened (post-audit)	4-of-5	1	4
Conservative	5-of-7	2	5
Maximum redundancy	4-of-7	3	4

`signatureThreshold = 3` of `M = 5` at launch tolerates two simultaneous signer outages or compromises before payout liveness is impaired. Raising the threshold (say to `4` of `5`) hardens the protocol against signer compromise but tightens the liveness window. Threshold changes are a single `setSignatureThreshold` call, gated by the `WouldBreachThreshold` invariant.

§ 12

Affiliate programme

`registerChallenge` accepts an `affiliate` parameter. When `affiliate != address(0)` and `affiliate != msg.sender` (else `SelfReferral`), the pool computes the affiliate's commission share via `_affiliateRateBps(affiliateMonthlyVolume[affiliate] [currentEpoch])` and transfers that share of the fee to the affiliate's wallet in the same transaction. Settlement is atomic; there is no separate claim function.

State

```
mapping(address => mapping(uint256 => uint256)) affiliateMonthlyVolume;
mapping(address => mapping(address => bool))    affiliateHasReferred;
uint256[] affiliateTierThresholds;   // ascending
uint256[] affiliateTierRates;        // each ≤ MAX_AFFILIATE_BPS, non-decreasing
uint256 constant MAX_AFFILIATE_BPS = 2000;
```

`currentEpoch = block.timestamp / 30 days`. The `affiliateMonthlyVolume[a][e]` mapping accumulates referred fee volume for affiliate `a` in epoch `e`. The window is sliding over time without explicit cleanup; the mapping is only read for the current epoch when computing a tier rate.

`affiliateHasReferred[affiliate][trader]` is set on first referral and consulted to gate the affiliate-rate path: a trader's first registration referred by an affiliate counts toward the affiliate's volume; subsequent registrations by the same trader do not, regardless of whether they pass `affiliate` again.

Tier evaluation

`_affiliateRateBps(monthlyVolume)` performs a linear search over `affiliateTierThresholds[]` and returns the corresponding rate from `affiliateTierRates[]`. Worst-case cost is `O(T)` per call where `T = affiliateTierThresholds.length`; in practice `T ≤ 5` and the cost is dominated by storage reads (warm slots after the first iteration).

Configuration

```
function setAffiliateTiers(uint256[] calldata thresholds,
                           uint256[] calldata rates) external onlyOwner
```

Pre-conditions:

- `thresholds.length == rates.length` (else `LengthMismatch`)
- `thresholds[]` strictly ascending (else `ThresholdsNotAscending`)
- `rates[]` non-decreasing (else `RatesNotNonDecreasing`)
- each `rate ≤ MAX_AFFILIATE_BPS` (else `RateExceedsCap`)

The owner can re-tier the ladder per call, but cannot raise an individual rate above the protocol-level cap without redeploying the implementation. Re-tierings do not retroactively re-rate already-registered challenges; they affect only future `registerChallenge` calls.

Diagram

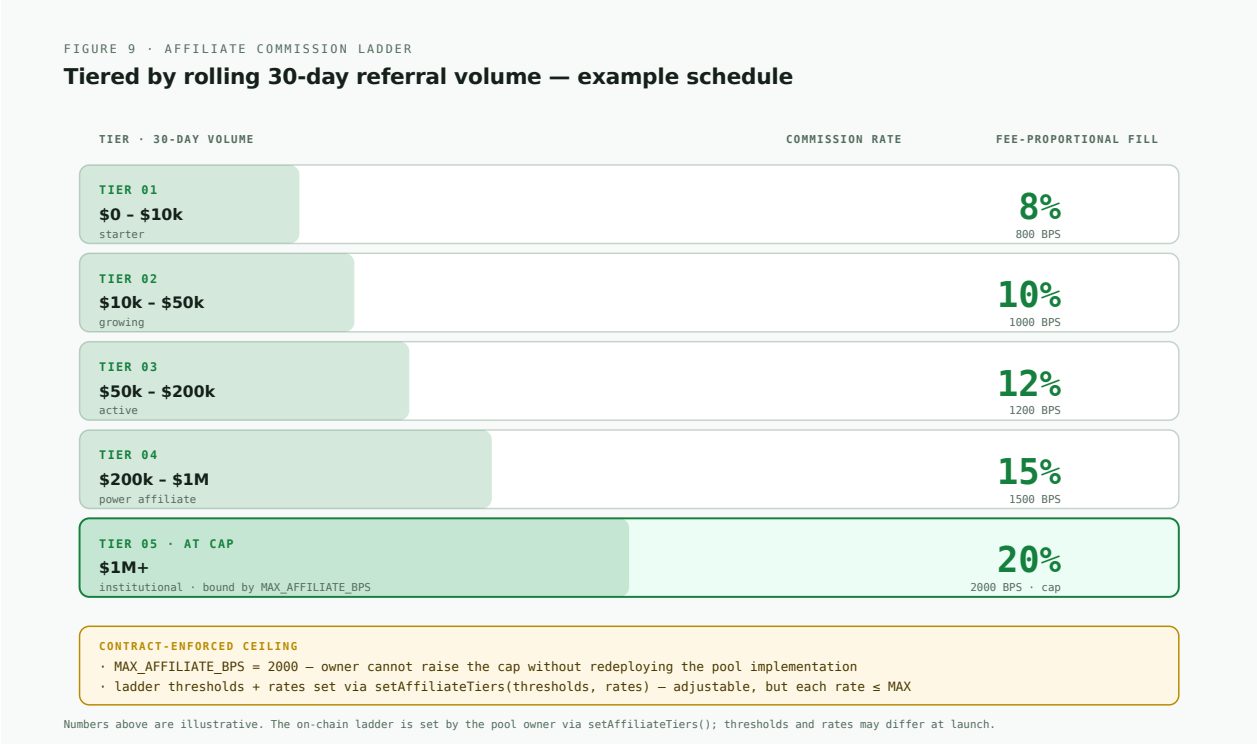


Figure 9 · Affiliate commission ladder · example schedule

§ 13

CFND token

`CFNDToken` is the LP share ERC-20. It is a thin wrapper over OpenZeppelin's `ERC20` and `Ownable`; `mint(to, amount)` and `burn(from, amount)` are restricted to `Ownable.owner()`, which is set to `ChainFundedPool`'s address at deployment.

Properties

PROPERTY	VALUE
Symbol	`CFND`
Decimals	`6` (matches USDC)
Owner / mint authority	`ChainFundedPool` proxy address (immutable post-deployment)
Initial NAV	`1.00 USDC per CFND` at first deposit
Transfer	standard ERC-20 `transfer` and `transferFrom`

Whitelisted profit-aware transfer

The pool maintains `cfndTransferWhitelist` (`mapping(address => bool)`) for staking endpoints — `CFGLPRewards` is the canonical example — that need to move CFND between holders while preserving per-CFND profit-accumulator correctness. Whitelisted endpoints call:

```
pool.transferCfnd(from, to, amount)
```

which (i) settles any pending profit for both `from` and `to` into their respective `rewardDebt` slots, then (ii) executes the transfer at the underlying ERC-20 layer. Direct ERC-20 `transfer` and `transferFrom` calls remain functional for non-whitelisted endpoints but bypass the accumulator synchronisation; staking integrations must use `transferCfnd` to avoid stranded yield.

`setCfndTransferWhitelist(addr, enabled)` is `onlyOwner` and emits `CfndTransferWhitelistUpdated`.

NAV invariant

The pool enforces `cfndToken.totalSupply() > 0` at the entry to `registerChallenge` (else `PoolMustHaveDeposits`). This rules out the degenerate state in which a trader pays a fee into a pool with no LP capital backing the eventual payout obligation. The first LP deposit creates the supply and unlocks challenge purchases; subsequent challenge fees flow into LP capital and LP yield without affecting `totalSupply`.

`cfndTotalSupply` changes only on:

- `deposit` → `mint(to, amount)`
- `withdraw` → `burn(from, amount)`

`claimProfit` does not change supply. `claimPayout` does not change supply. `transferCfnd` does not change supply (transfer between holders).

Why a fungible ERC-20 over an LP-NFT

LP positions in the pool are homogeneous: every CFND represents the same per-unit claim on capital and the per-share accumulator. Heterogeneous-position protocols (Uniswap V3 concentrated liquidity, etc.) require NFT positions to encode per-position parameters. ChainFunded's pool has no such per-position state; fungibility is the natural fit and removes special-casing in downstream integrations (staking, lending, secondary markets).

§ 14

CFG token & DAO

The CFG governance token, vesting schedule, distribution contracts, governance configuration, and treasury topology are out of scope for this document. They are the subject of a separate publication maintained at chainfundeddao.org.

The boundary is drawn deliberately: this document specifies the deterministic on-chain behaviour of `ChainFundedPool` and its satellites. Token-economic parameters and governance state are mutable under DAO action and have a different cadence of revision than the protocol surface itself.

Pointer

The DAO documentation covers, at minimum:

- `CFGToken` — supply, vesting cliffs, initial mint distribution
- `CFGVesting` — per-recipient vesting contracts (founder, contributor, investor)
- `CFGDistributor` — trader-rewards programme: CFG distributed per registered challenge, sourced from a fixed allocation
- `CFGLPRewards` — LP staking: CFND staked → CFG yield + USDC profit-pool yield via the whitelisted `transferCfnd` path
- `CFGGovernor` + `Timelock` — voting, proposal threshold, voting delay, voting period, execution delay
- Treasury composition, signer topology, and on-chain spend reporting

→ chainfundeddao.org

What this means for protocol-only readers

A reader who only requires the protocol's settlement and LP mechanics does not need to read the DAO publication. The protocol contracts are not dependencies of CFG; CFG-side contracts are dependencies of the protocol only via `cfndTransferWhitelist` (granting `CFGLPRewards` permission to move CFND on behalf of stakers). The grant is reversible by `setCfndTransferWhitelist(addr, false)`; the protocol continues to function with the whitelist empty.

§ 15

Risk surface

This section enumerates the protocol's risk surface by carrier: the trader, the LP, and shared risks borne by both via the contract.

Trader risk

RISK	MITIGATION / MECHANISM
Maximum monetary loss bounded to `feeAmount`	Simulated trading P&L cannot inflict additional cash loss. There is no on-chain margin or collateral call against the trader beyond the fee paid at `registerChallenge`.
Payout liveness — N signers must be online	N-of-M = 3-of-5 at launch tolerates two simultaneous outages. A funded trader's signed payout proof is valid until its `deadline`; signers may produce a proof at any time within the window without the trader losing standing.
Trading-engine fault that mis-records P&L	Out of scope of on-chain re-verification. Mitigation is operational: independent signer environments cross-check P&L derivations off-chain. A trading-engine fault that overstates trader profit produces an attested P&L the contract cannot detect.

LP risk

RISK	MITIGATION / MECHANISM
NAV depreciation below 1.00	Cumulative trader payouts that exceed (cumulative challenge fees + firm-cut LP slice) reduce LP capital. Bounded by `lpCapitalBalance() ≥ 0`; `claimPayout` reverts with `InsufficientLPCapital` once reserves are exhausted.
Withdrawal contention	All LP withdrawals draw from the shared USDC reserve. Concurrent large withdrawals near a pending payout can cause `claimPayout` to revert until fresh fees flow in. This is queue contention rather than insolvency, but is observable as payout latency.
Smart-contract risk on UUPS upgrade	`_authorizeUpgrade` is `onlyOwner`. Owner is a 2-of-2 multisig at launch (Labs + independent guardian). An emergency withdrawal by the owner is gated by `EMERGENCY_DELAY = 48 hours`. See §17.
Same-block deposit / withdraw cycling	Blocked by `lastDepositBlock[lp] == block.number` check on `withdraw`, reverting `SameBlockDeposit`.

Shared risk

RISK	MITIGATION / MECHANISM
USDC peg break or Circle blacklist of pool address	The pool supports only USDC. A peg break or address-level blacklist halts protocol functionality. Concentration is deliberate: supporting multiple stable assets would multiply contract surface and per-asset risk-management complexity.
Regulatory action against trader-evaluation accounts	External to the protocol. Jurisdictional access to the in-app trading engine is operational and may change. The on-chain protocol itself is permissionless to read and call; it cannot be unilaterally taken offline by a single jurisdiction.
Multi-signer compromise	A `signatureThreshold`-or-more compromise produces a valid signature aggregate. The on-chain re-verification (§4, §11) catches false rule attestations independent of signature validity; it does not catch a false P&L attestation that does not breach any rule. Mitigation in signer key-management: HSM custody, geographically distributed signing environments, post-hoc audit.
Owner key compromise	Owner is `Ownable2Step` — transfers require `acceptOwnership` from the new owner, blocking single-tx transfer attacks. The 48-hour emergency-withdrawal timelock provides a window during which a malicious owner action can be observed and LPs can withdraw.

§ 16

Security posture

The protocol's security model has four components: bounded on-chain surface, multi-signature signer set over payout state transitions, owner-only configuration with a 48-hour emergency-withdrawal timelock, and append-only storage layout across UUPS upgrades. Each is enforced in code rather than by operational policy.

Audit status

- **AI-driven security reviews** are finalised against the current implementations of `ChainFundedPool.sol`, `CFNDToken`, and the supporting CFG / vesting contracts. Findings have been remediated; the post-remediation codebase is the version deployed to Sepolia and slated for mainnet.
- **Independent third-party audit** is in progress as of mainnet preparation. The audit report will be published at the canonical URL on completion. Until then the operational posture is pre-third-party-audit: a TVL cap may be enforced operationally, and early participants are advised to size positions accordingly.

Owner + multisig topology

ROLE	HELD BY	POWERS
Pool owner (`Ownable.owner()`)	2-of-2 multisig (Labs key + independent guardian key)	`setChallengeTiers`, `setDefaultRules`, `setAffiliateTiers`, `addSigner` / `removeSigner`, `setSignatureThreshold`, `updateGuardian`, `updateLabsWallet`, `_authorizeUpgrade`, emergency withdrawal request / execute / cancel
Payout signer set	M=5 EOAs (3-of-5 threshold at launch), each in an isolated environment	Sign EIP-712 typed messages required by `claimPayout`, `registerPhase2`, `registerFunded`, `registerPostPayout`
Guardian	Single role, currently a Labs operational key	`pause()` (cannot `unpause`)

The guardian/owner asymmetry is deliberate: a single compromised guardian key can pause the protocol but cannot unilaterally resume it. `unpause` requires owner action.

Reentrancy + pause guards

All public state-changing entry points carry both `nonReentrant` (OpenZeppelin's `ReentrancyGuard`) and `whenNotPaused` (OpenZeppelin's `PausableUpgradeable`):

FUNCTION	NONREENTRANT	WHENNOTPAUSED
`deposit`	yes	yes
`withdraw`	yes	yes
`claimProfit`	yes	yes
`registerChallenge`	yes	yes
`registerPhase2` / `registerFunded` / `registerPostPayout`	yes	yes
`claimPayout`	yes	yes
owner config (`setChallengeTiers`, `setDefaultRules`, signer mgmt, etc.)	n/a	no — callable when paused so the owner can respond to an incident

UUPS upgradeability

`_authorizeUpgrade(address newImplementation)` is `onlyOwner`. Storage layout is append-only and the `contracts/storage-layout-baseline.json` file is checked against the to-be-deployed implementation by the deploy script. An upgrade that would shift any existing slot fails the check and cannot be deployed without an explicit baseline update.

Emergency withdrawal

```
requestEmergencyWithdrawal(to, amount, isCapital)
   → emits EmergencyWithdrawalRequested with executeAfter = block.timestamp +
EMERGENCY_DELAY

[ 48h elapses, observable by all participants ]

executeEmergencyWithdrawal(requestId)
   → moves USDC, emits EmergencyWithdrawalExecuted
```

`cancelEmergencyWithdrawal(requestId)` is available during the delay window and emits `EmergencyWithdrawalCancelled`. `EMERGENCY_DELAY = 48 hours`. The delay is the central operational guarantee: any withdrawal request is publicly observable for 48 hours before execution, providing LPs and external observers a window to react — withdrawing their own positions, alerting the community, or initiating governance counter-action against the owner.

§ 17

Roadmap

Q2 2026 — mainnet

- `ChainFundedPool` UUPS proxy deployment to Ethereum mainnet
- AI-driven security reviews finalised; third-party audit in progress
- Signer set initialised at `M = 5`, `signatureThreshold = 3`

Q3 2026 — audit, governance handover, APAC

- Third-party audit publication
- Operational TVL caps relaxed contingent on audit closure
- Affiliate ladder activated via `setAffiliateTiers` with on-chain referral attribution
- **Governance handover** — pool owner transfer from the launch 2-of-2 multisig to a Timelock-controlled governance contract via `Ownable2Step` (`transferOwnership` + `acceptOwnership`). Stage-1 governance is a Timelock + admin multisig pending CFG activation; Stage-2 swaps the admin role for `CFGGovernor` once the Q4 exchange launch puts CFG into circulation
- **CFG token contract deployed** alongside the governance Timelock. The token is on-chain but non-tradeable until Q4 — no liquidity, no exchange listings, vesting and distributor contracts paused
- **Singapore (APAC) customer service office** stood up to cover APAC trading hours, signer monitoring, and on-shore participant support. Off-chain function only — no contract surface change

Q4 2026 — CFG exchange launch

- **CFG public exchange listings** — token enters circulation via initial CEX / DEX listings (separate publication — see chainfundeddao.org)
- Vesting and distribution contracts activated against the now-circulating supply
- **`CFGGovernor` activation** — voting takes over from the admin multisig per Stage-2 of the Q3 handover. From this point, pool-owner state changes are gated by token-weighted on-chain votes through the existing Timelock

2027 — multi-asset

- Multi-asset collateral evaluation — adding additional stable collaterals expands contract surface and per-asset risk surface; subject to its own audit cycle and a separate amendment to this paper

---

References

- ChainFunded protocol source · github.com/Mezfinance/ChainFunded
- ChainFunded DAO documentation · chainfundeddao.org
- OpenZeppelin Contracts (v5) — `Ownable2StepUpgradeable`, `PausableUpgradeable`, `ReentrancyGuard`, `UUPSUpgradeable`, `SafeERC20`, `ECDSA`, `MessageHashUtils`
- EIP-712 — typed structured data hashing and signing
- ERC-20 — fungible token standard (USDC, CFND)
- Sushi MasterChef — per-token reward accumulator pattern used by `accProfitPerToken` / `rewardDebt`

Disclosures

ChainFunded is software. It is not a regulated financial product and this document is not investment advice. Trading evaluation accounts is high-risk; participants may lose the fees they pay.

The protocol is in an early operational phase. TVL caps, signer-set parameters, fee splits exposed via constants, challenge tier prices, and governance topology may change with operational decision or DAO action. The on-chain source is the canonical specification; this document describes the protocol as deployed at the version stated on the cover.

Participants should review the on-chain contract source directly before participating, and should consult their own counsel where applicable. Past performance of the simulated trading environment, of the LP profit pool, or of any referenced third-party prop firm is not predictive of future outcomes.

§ 18

Appendix A · Threat model

Enumerates the adversaries the protocol's design takes seriously, the capabilities each holds, and the on-chain mechanisms that constrain them. Complements the risk-by-carrier framing of §16 — this appendix is adversary-by-adversary, which is the framing an auditor or security reviewer is most likely to use.

A · Malicious trader

Capability. Any EOA can pay an evaluation fee and trade. A malicious trader optimises trade execution against the rule set, or attempts to extract funds beyond the fee paid.

ATTEMPT	CONSTRAINED BY
Withdraw funds without satisfying rules	`claimPayout` re-verifies every rule snapshotted at registration against the asserted P&L. Any breach → named `RuleBreached*` revert (§11).
Submit a stale or already-claimed payout proof	`settledPayouts[payoutId]` flag is set on success → `PayoutAlreadySettled` on replay. `deadline` field checked against `block.timestamp` → `SignatureExpired`.
Forge signatures	`ECDSA.recover` on every signature, each must be in `authorizedSigners[]` → `UnauthorizedSigner`.
Register a free / zero-fee challenge to bypass economic cost	`feeAmount` must equal `challengeTierFee[accountSize]` exactly → `TierNotEnabled` otherwise.

Residual. Maximum monetary loss to the protocol from a malicious trader is bounded by zero — the trader can never extract more than they could legitimately earn under the snapshotted rules.

B · Malicious LP

Capability. Any EOA can deposit USDC and receive `CFND`. A malicious LP attempts to game the profit accumulator, grief withdrawal liveness, or extract more than their pro-rata share.

ATTEMPT	CONSTRAINED BY
Deposit-then-withdraw in a single block to capture accrued yield without taking risk	`lastDepositBlock[lp] == block.number` check on `withdraw` → `SameBlockDeposit` revert.
Withdraw a CFND amount they don't hold	Standard ERC-20 balance check → `InsufficientCFND`.
Front-run a `claimPayout` to drain capital before settlement	All paths through the pool are `nonReentrant` and `whenNotPaused`; LP withdrawals and claim-payouts are serialised by the EVM's single-tx model. Withdrawal contention exists but does not exceed pro-rata.
Manipulate `accProfitPerToken` accumulator	Accumulator is internal state updated by the pool on every profit-pool credit; LP-callable functions only read it via `rewardDebt` reconciliation. No external setter.

Residual. LP cannot extract more than their `cfndAmount × NAV` plus their pending profit. NAV is determined entirely by pool state they cannot directly write.

C · Compromised signer subset (< threshold)

Capability. Attacker controls fewer than `signatureThreshold` of the `M` authorised signer keys.

ATTEMPT	CONSTRAINED BY
Forge a valid payout signature aggregate	`signatures.length >= signatureThreshold` check → `InsufficientSignatures`. Cannot meet threshold with fewer keys.
Submit a partial signature set with replays of the same compromised key	Signatures must be sorted ascending by recovered signer address with no duplicates → `DuplicateOrUnsortedSignature`.
Coerce a non-compromised signer to sign	Off-chain. Mitigation: independent signer environments, HSM key custody.

Residual. No on-chain harm. Compromise of fewer than threshold keys is operationally observable (signature flow stops working) and rotatable (`removeSigner(addr)` then `addSigner(newAddr)`).

D · Compromised signer set (≥ threshold)

Capability. Attacker controls at least `signatureThreshold` keys and can produce valid signature aggregates over arbitrary typed messages.

ATTEMPT	CONSTRAINED BY
Sign a payout for an account that breached a rule	`claimPayout` re-verifies all rules against the snapshotted challenge state. The signature is valid; the rule check fails independently → `RuleBreached*` revert (§4, §11).
Sign a payout for an account whose registration the attacker fabricated	The pool's `challenges[accountId]` mapping is only writable through `registerChallenge` / `adminRegisterChallenge`. Fabrication requires either a fee payment (no economic gain) or admin-key compromise (separate threat).
Sign a P&L that consistent with the snapshotted rules but doesn't match the trader's actual trades	**Residual risk.** The on-chain check has no view of the trade ledger; a false P&L that satisfies the rule set is a valid payout from the contract's perspective. Mitigation is operational (per-signer independent replay) and detective (post-hoc audit of off-chain attestations against trade history).

Residual. A coordinated full-set compromise can extract a single fraudulent payout of any size within `lpCapitalBalance()`. Mitigation is in signer-key custody discipline (HSM, geographic distribution, distinct operational environments) and in raising the threshold post-launch.

E · Compromised pool owner

Capability. Attacker controls the `Ownable2Step` owner key (2-of-2 governance multisig at launch).

ATTEMPT	CONSTRAINED BY
Drain capital via emergency withdrawal	`requestEmergencyWithdrawal` → `EmergencyWithdrawalRequested` event → 48h `EMERGENCY_DELAY` → `executeEmergencyWithdrawal`. LPs and external observers see the request and have 48 hours to withdraw their own positions.
Upgrade pool to a malicious implementation	`_authorizeUpgrade` is `onlyOwner`. The deploy pipeline checks `storage-layout-baseline.json` and aborts on any non-appending change, blocking the obvious "swap implementation to drain funds" path. Even with an upgrade, the new implementation must be deployed and the upgrade tx is publicly observable before LPs need to react.
Single-tx ownership grab from a leaked old key	`Ownable2Step` requires `acceptOwnership()` from the new owner — single-key transfer attacks fail.
Adjust tier fees, rules, or signer set to extract value	Owner can update `challengeTierFee`, `setDefaultRules`, `addSigner` etc. These changes do not affect already-registered challenges (rules are snapshotted at registration) and are publicly logged. No path to retroactive rule change.
Disable `paused` indefinitely after malicious pause	`unpause` is `onlyOwner`, but `pause` itself is callable by either owner or guardian. The pause asymmetry means a single compromised guardian can stop the protocol but cannot keep it stopped past governance action.

Residual. A complete owner-key compromise can attempt an emergency withdrawal; the 48-hour timelock is the LP's defence window. Future migration to a `CFGGovernor` + `Timelock` (§18, Q3 2026) extends this from a 2-of-2 multisig to token-weighted governance.

F · External attacker (no privilege)

Capability. Any address with no protocol-level role.

ATTEMPT	CONSTRAINED BY
Reentrancy into state-changing functions	OpenZeppelin `ReentrancyGuard` on every public state-changing entry point.
Manipulate state during a pause	`whenNotPaused` modifier blocks deposit / withdraw / claim / register paths during pause. Owner config calls remain available for incident response.
Drain funds via smart-contract bug	Mitigated by audit posture: AI audits finalised, full third-party audit in progress (§17). The protocol's surface is a single core contract — concentration is deliberate to make this mitigation tractable.
Front-run a profit claim to extract LP yield	`claimProfit` operates only on the calling LP's `rewardDebt` and balance — there is no MEV-extractable state in the call.

Residual. Smart-contract risk is the load-bearing mitigation gap until the third-party audit publishes. TVL caps are operationally enforced during the pre-audit phase to bound exposure.

§ 19

Appendix B · Notation reference

A glossary of the typed conventions the document and the contracts use. Grep aid for readers cross-referencing the on-chain source.

Units

UNIT	MEANING
`USDC`	Circle USD Coin, 6 decimals on Ethereum mainnet. Every USDC figure in this paper is in human units (`$50.00`); on-chain the same value is `50_000_000` (`50 * 10^6`).
`BPS` (basis points)	One ten-thousandth of a whole. `BPS = 10_000` is the contract constant. `2400 BPS = 24%`. Percent-of-percent calculations multiply BPS values then divide by `BPS`.
`seconds`	Unix timestamp, no time zone. `block.timestamp` is the canonical clock for `deadline` checks.
`wei`	Solidity native — used only for ETH gas reimbursement amounts (`gasReimbursementUsdc` is in USDC units, not wei).

Identifier conventions

CONVENTION	EXAMPLE	WHERE USED
`UPPER_SNAKE_CASE`	`FEE_PROTOCOL_BPS`, `FEE_PROFIT_POOL_BPS`, `EMERGENCY_DELAY`	Solidity `constant` and `immutable` values. Implementation-fixed; changing requires a UUPS upgrade.
`camelCase`	`traderShareBps`, `accountId`, `gasReimbursementUsdc`	Function parameters and local variables.
`camelCase` (state)	`labsWallet`, `signatureThreshold`, `accProfitPerToken`	Public storage variables. Readable from any block via the auto-generated getter.
`camelCase()`	`claimPayout()`, `claimProfit()`, `registerChallenge()`	Function references. Parentheses indicate this is a function, not a state slot.
`accountId` (24-char string)	`"a1b2c3d4e5f6789012345678"`	All challenge identifiers. Length is enforced (`bytes(accountId).length == 24`) at registration.
`payoutId` (variable string)	any non-empty string	Per-payout replay-protection key. Set by the backend, length-unconstrained, must be unique per `claimPayout`.
`bytes32 indexed`	event topic	Indexed event field — searchable on Etherscan via log filter.

Math / formula style

STYLE	EXAMPLE	USE CASE
Code-block formula	`NAV = (totalUSDC − profitPool) / cfndTotalSupply`	Core protocol formulas. Always uses the on-chain state names.
Inline backtick	`(feeAmount × FEE_PROTOCOL_BPS) / BPS = 24%`	Per-line ratio derivations inside prose.
Table row	tier · rate · destination	When a formula has more than two parameters or branches by condition.
Comparison operators	`≥` `≤` `=` `>` `<`	Standard Unicode, not the ASCII `>=` etc.

Cross-reference style

FORM	MEANING
`§N`	Section N of this paper (e.g. `§11` = Payout enforcement).
`Fig. N`	Figure N (e.g. `Fig. 6` = fee split diagram).
`(§4, §11)`	Multiple section references inside a parenthetical.
`[link text](url)`	External link, always rendered in primary green.

Token symbols

SYMBOL	SPELLING	WHEN IN BACKTICKS
USDC	bold, no backticks in prose; `USDC` with backticks when referring to the contract identifier or an `IERC20(USDC)` cast	
CFND	bold in prose, `CFND` in backticks when referring to the `CFNDToken` contract or the symbol as a code identifier	
CFG	bold in prose, `CFGToken` in backticks for the contract	
ETH	bold in prose, never in backticks — never used as a code identifier	

Address style

FORM	MEANING
`0xfbdc2592f7d8ae59d1abebb88cf33b2ac70383fc`	Full mainnet address, lowercased for hash-friendliness.
`0xfbdc…83fc`	Truncated address — first 6 + last 4 chars, joined by an ellipsis. Used in prose where the full address would be visually heavy.
`address(0)`	The zero address. Used in contract conditions like `if (affiliate != address(0))`.

Error names

Custom errors are referenced inline as bare identifiers — e.g., `RuleBreachedDailyLoss`, `PayoutAlreadySettled`, `InsufficientSignatures`. They are not prefixed with the contract name; the contract context is implicit from the calling function referenced in the same sentence.

The complete list of errors is in `contracts/contracts/ChainFundedPool.sol` lines 149–213.